6. Jenkins를 활용한 CI/CD 환경 구성 - 운영 서버(원격 서버) 구성 및

Git, Git hub Web Hook, Jenkins, Spring 서버 연동을 통한 CI/CD 자동 배포 실습입니다.

1. 운영서버(원격 서버) 생성 및 설정

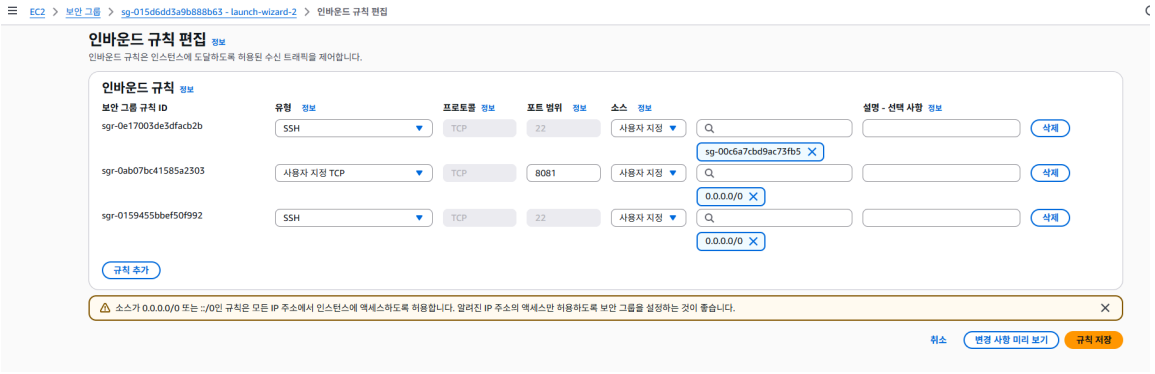
1. AWS EC2 인스턴스 생성

- EC2 > 인스턴스 > 인스턴스 시작
- 이름 : spring
- 애플리케이션 및 OS Image : Amazon Linux, Amazon Linux 2023 AMI
- 인스턴스 유형 : t2.micro
- 키 페어(로그인) : 새로 생성
  - 키 페어 이름 : spring-rsa
  - 키 페어 유형 : RSA
  - 프라이빗 키 파일 형식 : .ppk
  - 키 페어 생성 클릭
- 네트워크 설정 : 보안 그룹 생성
- 나머지는 선택된 상태로 인스턴스 시작

2. 네트워크 설정

- EC2 인바운드 규칙 추가 : 8081 포트 추가(스프링 서버)
  - spring 인스턴스 선택
  - 보안 - 보안 그룹 클릭
  - 인바운드 규칙 편집
  - 규칙 추가
- EC2 인바운드 규칙 추가 : jenkins 서버 SSH 포트 추가

- spring 인스턴스 선택
- 보안 - 보안 그룹 클릭
- 인바운드 규칙 편집
- ssh 유형 선택
- jenkins 보안 그룹 선택 및 규칙 저장



- 탄력적 IP 생성 및 연결

3. docker 설치

- spring-server 인스턴스(remote-server)에 docker 설치(Ansible을 이용하면 docker도 자동으로 설치되도록 가능)
- putty로 접속
  - host name 입력(ec2-user@{spring-server 탄력적ip})
  - connection > ssh > auth > credentials 선택
  - Private key file for authentication → spring-rsa.ppk 파일 선택
  - session에서 saved session
  - open 클릭하여 접속
- 아래 명령어 실행

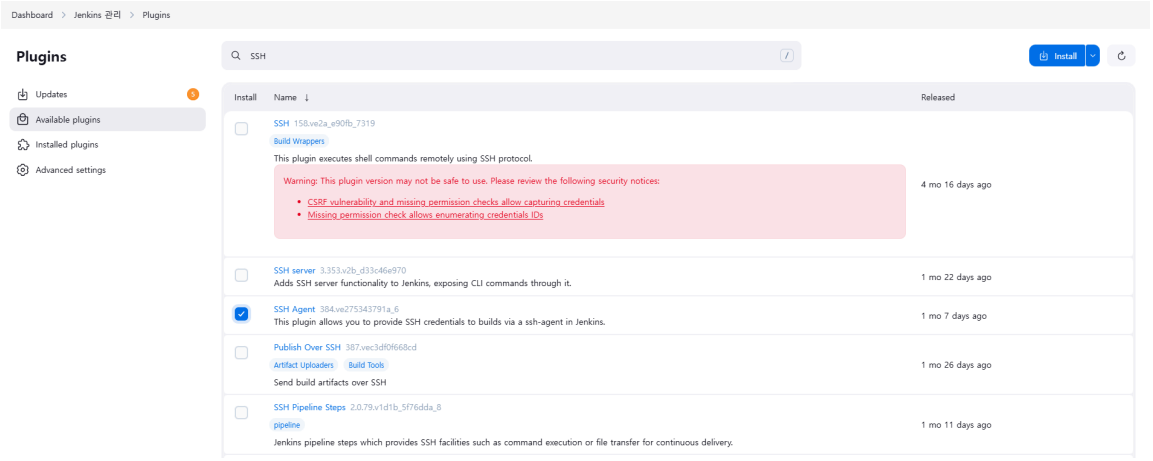
```
# 시스템의 모든 패키지를 최신 버전으로 업데이트
$ sudo dnf update -y

# Docker 패키지를 설치
```

```
$ sudo dnf install docker -y

# Docker 서비스를 시작 (시스템 부팅 시 자동으로 실행되지는 않음)
$ sudo systemctl start docker

# 시스템 부팅 시 Docker 서비스가 자동으로 실행되도록 설정
$ sudo systemctl enable docker

# 현재 사용자를 'docker' 그룹에 추가 (sudo 없이 docker 명령어를 사용하기 위함)
$ sudo usermod -aG docker $USER

# 현재 세션에서 변경된 그룹 권한을 적용 (로그아웃/로그인 대신 즉시 반영)
$ newgrp docker

# 설치된 Docker 버전 확인
$ docker -v
```

2. Jenkins 와 Remote 서버(Spring) 연결

1. jenkins + spring-server SSH 연결

- jenkins 페이지 접속 및 로그인
- SSH Agent Plugin
  - `SSH Agent Plugin`은 **Jenkins에서 SSH 키 기반 인증을 안전하게 다룰 수 있도록 도와주는 플러그인**입니다. 이 플러그인을 사용하면, Jenkins 파이프라인이나 Job 내에서 **SSH private key를 안전하게 불러와서 외부 서버에 접속**할 수 있게 됩니다.
- SSH Agent Plugin 설치
  - 설정 > Jenkins 관리 > 플러그인 > 이용 가능 플러그인 > SSH Agent 플러그인 설치



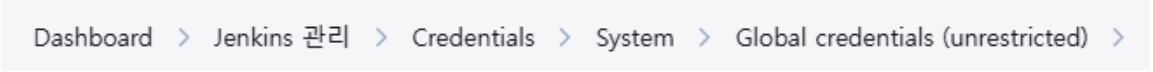
- jenkins-server의 ssh public key를 spring-server의 .ssh/authorized_keys 파일에 추가
  - jenkins-server rsa key 복사

```
cd /var/lib/jenkins/.ssh
cat jenkins_rsa_key.pub
ssh-rsd ~~~ key값 복사
```

  - spring-server authorized_key에 rsa public key 복사

```
echo "ssh-rsa AAAAB3NzaC1yc2EAAAABIwAAAQEA..." >> ~/.ss
h/authorized_keys
```

  - Global credentials 등록 → Git 연결 작업 시 이미 생성하였으므로 생략



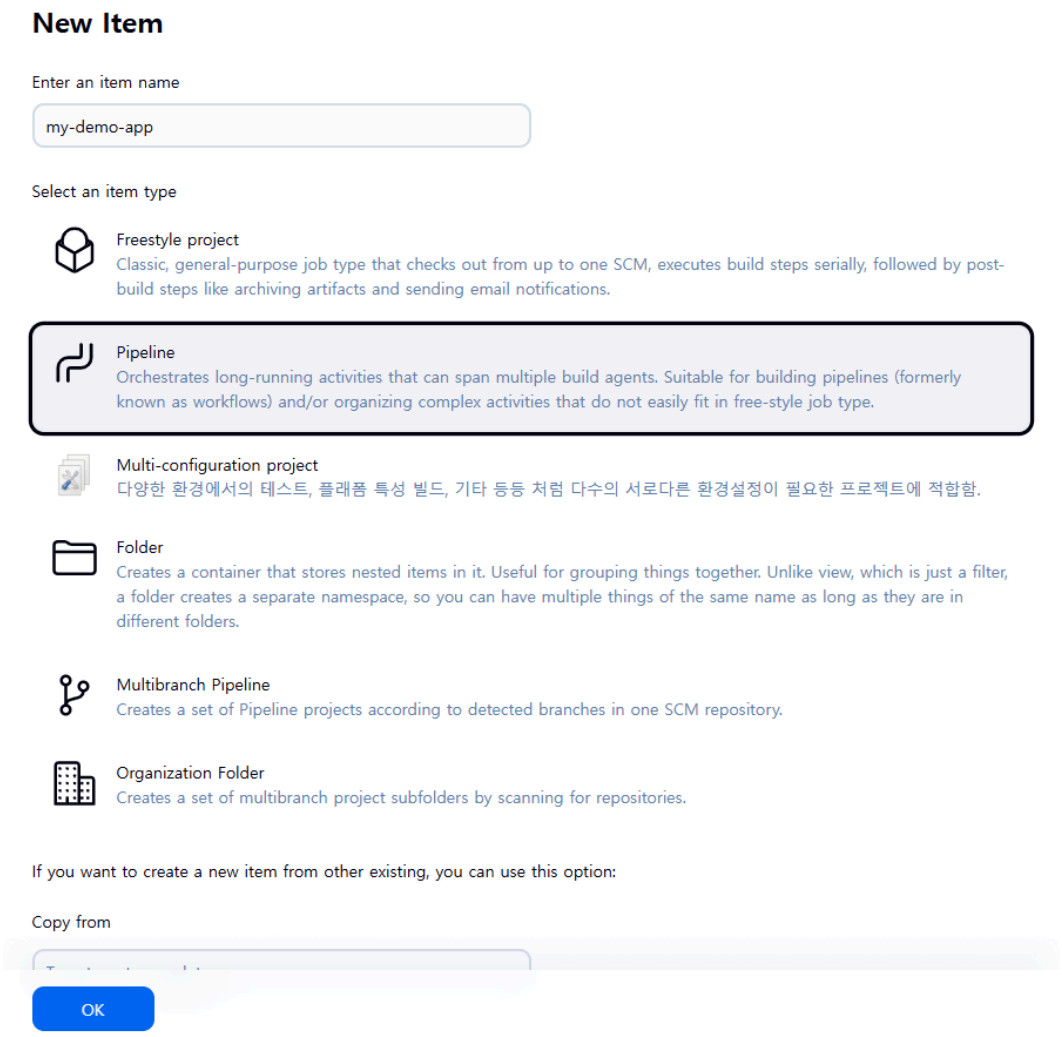
  - 서버 재시작하여 설정 적용

```
sudo reboot
```

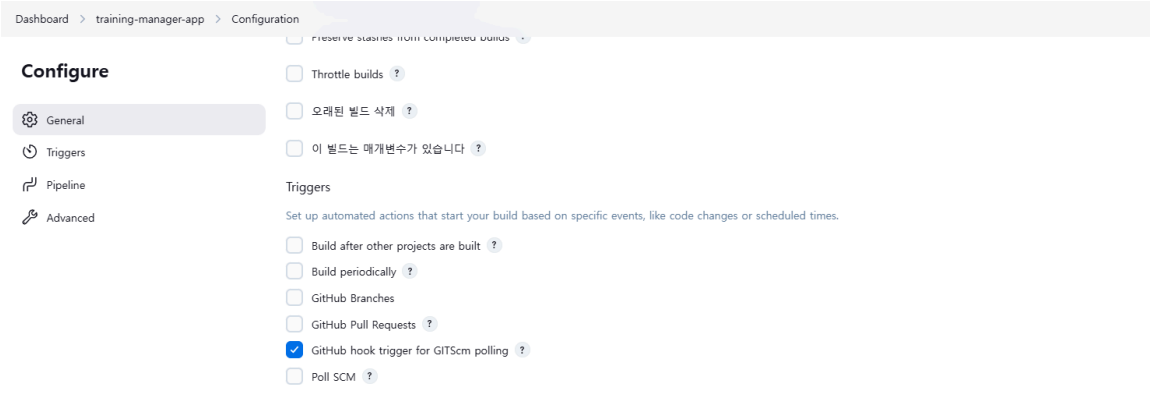
3. Jenkins + git hub web hook 연결

- jenkins 관리 > Plugins > Available plugins : github integration 설치

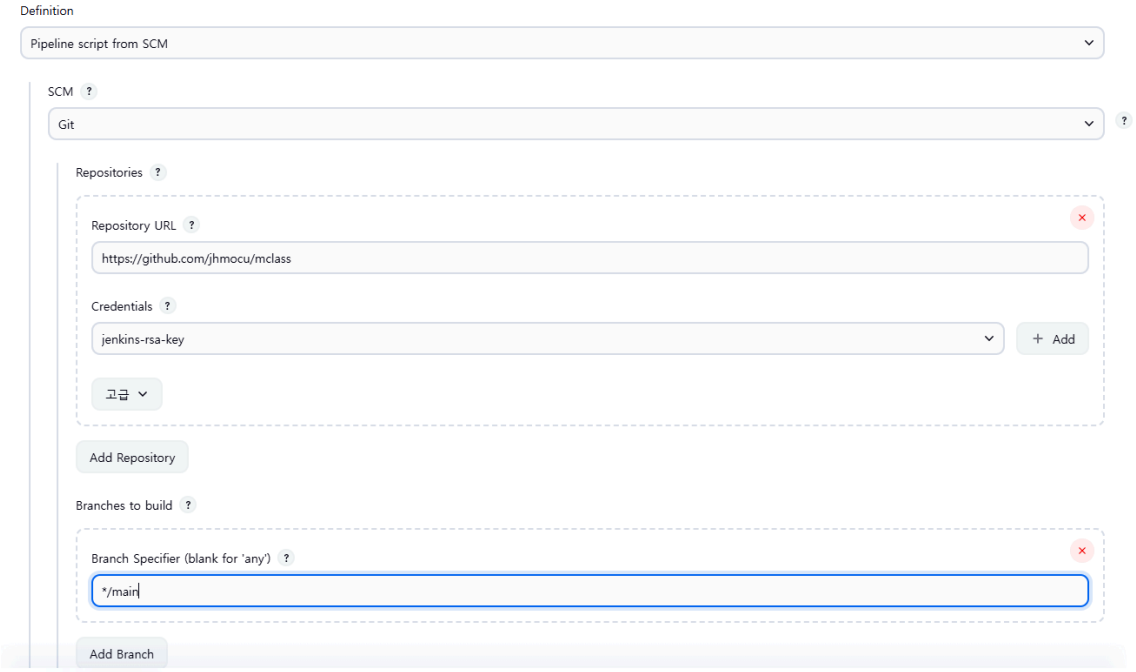
- New Item - Pipeline 선택 및 item 생성



- 생성된 item > Configuration
  - General - Github project 연결(HTTPS 주소)
    - https://github.com/jhmocu/mclass
  - Triggers - GitHub hook trigger for GITScm polling 선택

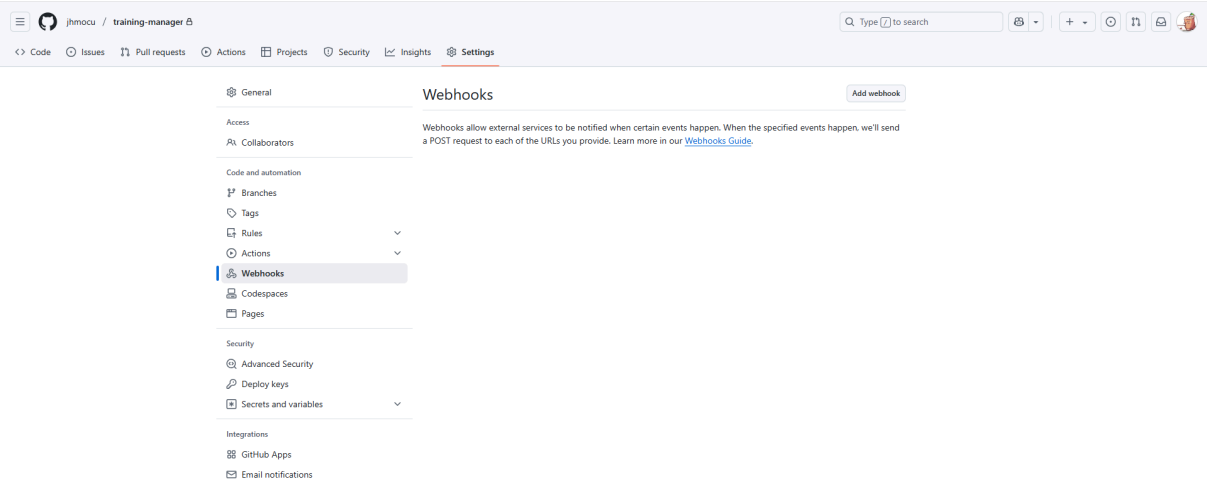


- pipeline 정보 입력

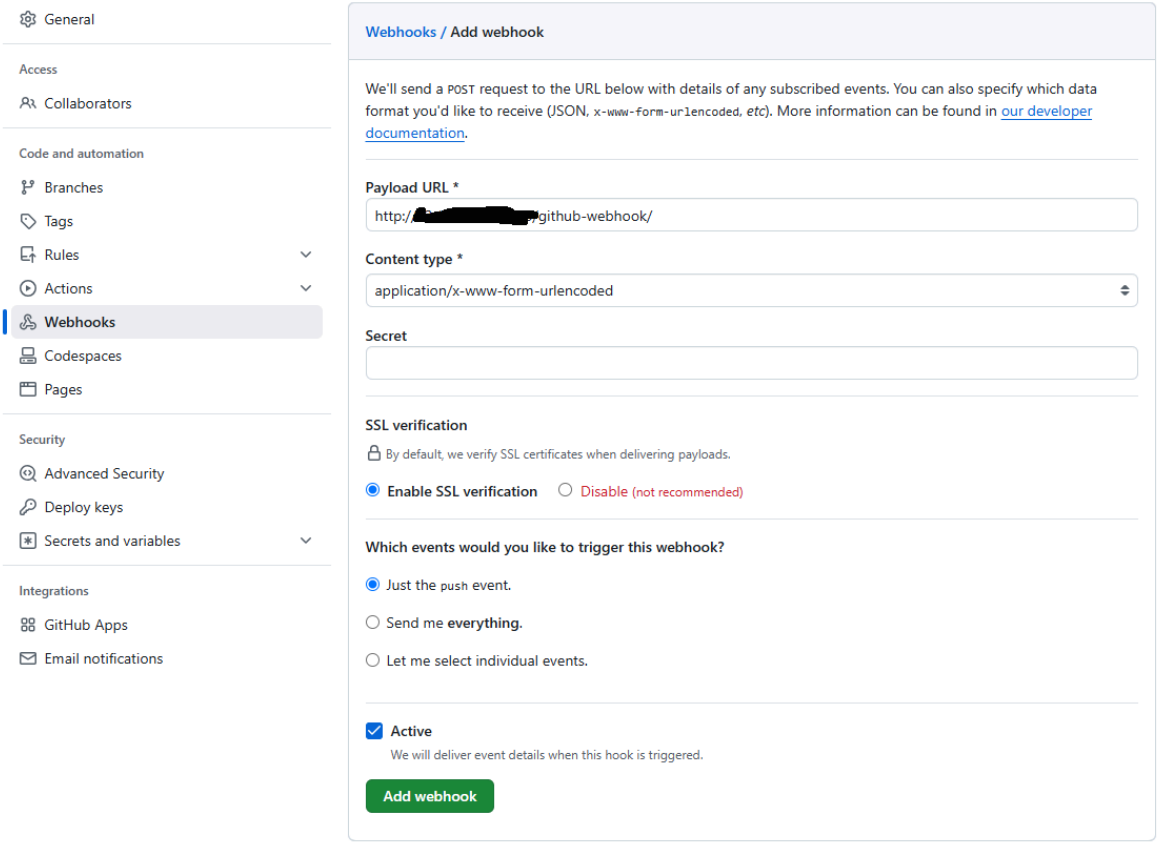


- Save 클릭

- git hub 접속 - 연결할 Repository 선택 → Setting → WebHooks → Add webhook



- 정보 입력(http:/{Jenkins URL}:{port}/github-webhook/)
  - http://13.125.207.231:8080/github-webhook/



4. Docker File, Jenkins File 생성

1. Dockerfile

- Docker File이란
  - Docker 이미지를 만들기 위한 설정 스크립트 파일
  - Dockerfile —▶️ docker build —▶️ Docker 이미지 —▶️ docker run —▶️ 컨테이너
- 장점
  - 동일한 환경설정 : OS, JDK, 설정 등을 통일된 환경으로 만들 수 있음
  - 배포 자동화 : CI/CD에서 자동으로 이미지 생성 & 배포 가능
  - 코드 관리 : Git에서 Dockerfile도 함께 버전 관리
- 주요 Dockerfile 명령어

명령어	설명
FROM	베이스 이미지를 지정 (예: ubuntu, openjdk, node 등)
WORKDIR	컨테이너 내에서 작업할 디렉토리 지정
COPY	로컬 파일을 컨테이너로 복사
ADD	`COPY`와 비슷하지만, 압축 파일 자동 해제 및 URL 다운로드 가능
RUN	이미지 빌드 시 실행할 명령어 (예: 패키지 설치)
CMD	컨테이너 실행 시 기본으로 실행되는 명령어 (1개만 존재)
ENTRYPOINT	실행 시 고정된 명령어 지정 (CMD와 함께 사용 가능)
EXPOSE	컨테이너에서 사용하는 포트 지정
ENV	환경 변수 설정
VOLUME	외부 저장소(호스트)와 데이터 공유 디렉토리 지정

- Spring 프로젝트에 Dockerfile 생성
  - Dockerfile

```
FROM eclipse-temurin:17-jdk

# JAR 파일이 저장될 작업 디렉토리 설정
WORKDIR /app

# Maven 또는 Gradle 빌드 후 생성된 JAR 파일을 컨테이너 내부 /app 디렉토리에 app.jar 이름으로 복사
COPY app.jar app.jar
```

```
# 4. 실행 포트 지정
EXPOSE 8081

# 5. 컨테이너 실행 시 JAR 실행
ENTRYPOINT ["java", "-jar", "app.jar"]
```

2. Jenkinsfile

- Jenkinsfile이란
  - Jenkinsfile은 Jenkins에서 자동 빌드, 테스트, 배포 작업을 코드로 정의한 파일
  - 보통 Git 저장소에 Jenkinsfile이라는 이름으로 저장되어 Jenkins가 이를 읽어 자동으로 작업을 실행
  - **Pipeline**은 소프트웨어 개발 과정(빌드, 테스트, 배포 등)을 **연속적 자동화(CI/CD)** 하기 위한 Jenkins의 기능
  - **pipeline / agent / environment / tools** (환경 정의)
  - **stages / stage / steps** (단계별 실행)
  - **post** (결과 처리)
- 장점
  - 코드 기반 관리 - 빌드/배포 과정을 코드로 관리할 수 있어 형상 관리 가능
  - 자동화 - Git Push만 해도 자동으로 빌드 & 배포 가능
  - 협업에 유리 - 빌드/배포 로직도 코드 리뷰 대상이 됨
  - 반복 줄이기 - 매번 수동 설정 없이 모든 작업이 자동화됨
- 주요 명령어

키워드	설명
pipeline { }	Jenkins 파이프라인 전체 구조
agent	실행할 노드/환경 지정 (`any`, `none`, 특정 라벨, Docker 등)
environment { }	환경 변수 설정
tools { }	Maven, JDK 같은 도구 지정
stages { }	여러 개의 단계(stage)를 그룹화
stage('이름') { }	빌드, 테스트, 배포 단계 구분

키워드	설명
steps { }	stage 안에서 실행할 실제 명령어
script { }	Groovy 스크립트 사용 가능
sh	셸 명령어 실행 (리눅스/유닉스)
bat	배치 명령어 실행 (윈도우)
echo	로그 출력
input	사용자 입력 대기 (승인 단계에서 사용)
when { }	특정 조건에서만 실행 (예: 브랜치 체크)
post { }	빌드 후 처리 (성공, 실패, 항상 실행 등)

- Spring 프로젝트에 Jenkinsfile 생성
  - Jenkinsfile

```
pipeline {
    agent any // 어떤 에이전트(실행 서버)에서든 실행 가능

    tools {
        maven 'maven 3.9.12' // Jenkins에 등록된 Maven 3.
9.12을 사용
    }

    environment {
        // 배포에 필요한 변수 설정
        DOCKER_IMAGE = "demo-app"   // 도커 이미지 이름
        CONTAINER_NAME = "springboot-container" // 도커
컨테이너 이름
        JAR_FILE_NAME = "app.jar"   // 복사할 JAR 파일 이
름
        PORT = "8081"   // 컨테이너와 연결할 포트

        REMOTE_USER = "ec2-user"    // 원격(spring) 서버
사용자
        REMOTE_HOST = "13.125.189.5"    // 원격(spring)
서버 IP(Public IP)

        REMOTE_DIR = "/home/ec2-user/deploy"    // 원격
서버에 파일 복사할 경로
```

```
        SSH_CREDENTIALS_ID = "e365c3f6-3d96-4647-923c-a5
8e121b99f7" // Jenkins SSH 자격 증명 ID
    }

    stages {
        stage('Git Checkout') {
            steps { // step : stage 안에서 실행할 실제 명령
어
                // Jenkins가 연결된 Git 저장소에서 최신 코드
체크아웃
                checkout scm
            }
        }

        stage('Maven Build') {
            steps {
                // 테스트는 건너뛰고 Maven 빌드
                sh 'mvn clean package -DskipTests'
                // sh 'echo Hello' : 리눅스 명령어 실행
            }
        }

        stage('Prepare Jar') {
            steps {
                // 빌드 결과물인 JAR 파일을 지정한 이름(app.j
ar)으로 복사
                sh 'cp target/demo-0.0.1-SNAPSHOT.jar
${JAR_FILE_NAME}'
            }
        }

        stage('Copy to Remote Server') {
            steps {
                // Jenkins가 원격 서버에 SSH 접속할 수 있도
록 sshagent 사용
                sshagent (credentials: [env.SSH_CREDENTI
ALS_ID]) {
                    // 원격 서버에 배포 디렉토리 생성 (없으면
```

```
새로 만듦)
                    sh "ssh -o StrictHostKeyChecking=no
-o UserKnownHostsFile=/dev/null ${REMOTE_USER}@${REMOTE_
HOST} \"mkdir -p ${REMOTE_DIR}\""
                    // JAR 파일과 Dockerfile을 원격 서버에
복사
                    sh "scp -o StrictHostKeyChecking=no
-o UserKnownHostsFile=/dev/null ${JAR_FILE_NAME} Dockerf
ile ${REMOTE_USER}@${REMOTE_HOST}:${REMOTE_DIR}/"
                }
            }
        }

        stage('Remote Docker Build & Deploy') {
            steps {
                sshagent (credentials: [env.SSH_CREDENTI
ALS_ID]) {
                            sh """
ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/d
ev/null ${REMOTE_USER}@${REMOTE_HOST} << ENDSSH
    cd ${REMOTE_DIR} || exit 1
    docker rm -f ${CONTAINER_NAME} || true
    docker build -t ${DOCKER_IMAGE} .
    docker run -d --name ${CONTAINER_NAME} -p ${POR
T}:${PORT} ${DOCKER_IMAGE}
ENDSSH
                                        """
                }
            }
        }

    }

}
```

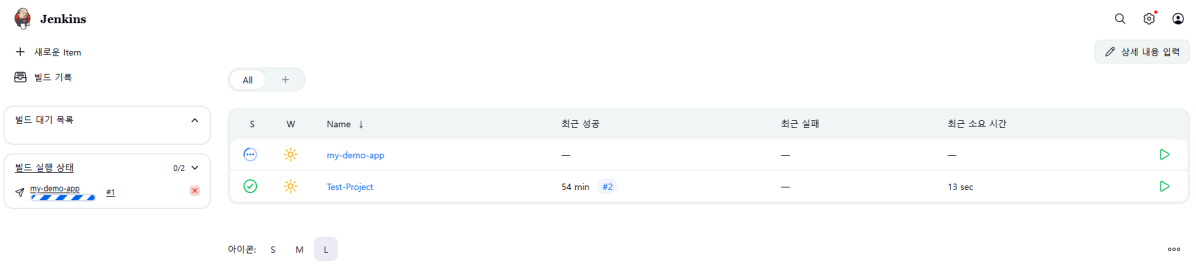
```
        stage('Remote Docker Build & Deploy') {
            steps {
                sshagent (credentials: [env.SSH_CREDENTIALS_ID]) {
        sh """
ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null ${REMOTE_USER}@${REMOTE_HOS
    cd ${REMOTE_DIR} || exit 1
    docker rm -f ${CONTAINER_NAME} || true
    docker build -t ${DOCKER_IMAGE} .
    docker run -d --name ${CONTAINER_NAME} -p ${PORT}:${PORT} ${DOCKER_IMAGE}
ENDSSH
"""
                }
            }
        }
```

- ENDSSH 공백 문제 해결하기

- Git 명령어로 commit 실행

```
git add .
git commit -m 'add code'
git push origin main
```

- 설정이 성공하면 git commit하면 자동으로 Jenkins에서 배포 수행



- 배포 확인 후 Spring server에 접속
- 접속 url - 원격서버IP:8081
- 문제가 있을 경우 모든 인스턴스 재시작